



Flash-based SSD

어떻게 만들고, 무엇이 문제이며, 어떻게 해결하는가

서혜원 2466071



SSD란 무엇인가?

→ solid-state storage

HDD

- 기계식 회전 디스크
- 느리고 충격에 약함
- 전원 off → 데이터 유지

SSD

- 트랜지스터로만 구성
- 빠르고 충격에 강함
- 전원 off → 데이터 유지

Flash 메모리란?

→ 트랜지스터에 전하를 가두는 방식으로 데이터를 저장하는 비휘발성 반도체 메모리

- 전원 off → 데이터 유지
- 영구저장 가능
- Solid-state (기계부품 X)

오늘 다룰 SSD는 모두 **NAND-based Flash**

- 대용량
- 저렴
- 빠른 쓰기
- 용도: SSD, USB, 스마트폰

Flash 내부구조

Bank : 플래시 칩 전체 구획

Block : 128KB~256KB, Erase 단위

Page : 4KB, Read/Program 단위

Cell : SLC, MLC, TLC

cell 종류: SLC, MLC, TLC

SLC

Single-Level Cell

저장: **1bit**

성능: 가장 빠름

수명: 100,000 P/E cycle

가격: 가장 비쌈

용도: 서버, 산업용

MLC

Multi-Level Cell

저장: **2bit**

성능: 중간

수명: 10,000 P/E cycle

가격: 중간

용도: 소비자용 SSD

TLC

Triple-Level Cell

저장: **3bit**

성능: 가장 느림

수명: ~3,000 P/E cycle

가격: 가장 저렴

용도: 대용량 저장장치

Flash의 3가지 기본 동작

Read

- Page 단위
- 속도: 수십 μ s
- Random Access Device

Program (Write)

- Page 단위
- 속도: 수백 μ s
- 반드시 Erase이후에 가능

Erase

- Block 단위
- 속도: 수ms
- 방법: 블록 내 모든 bit 1로 초기화

SSD 내부 구조

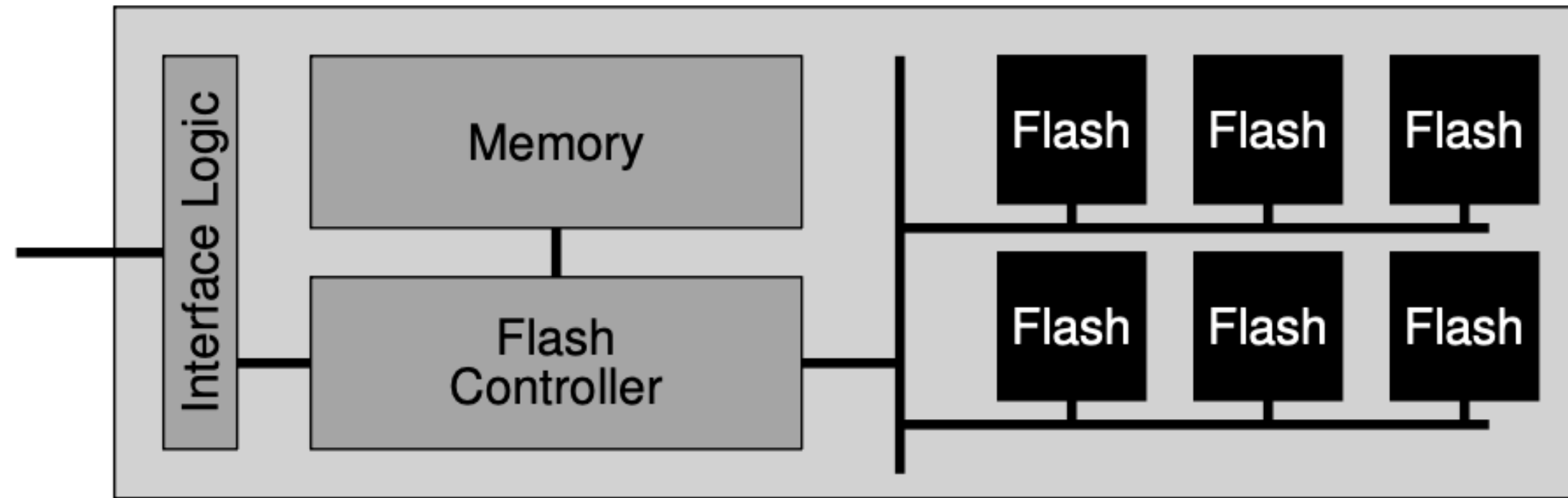


Figure 44.3: A Flash-based SSD: Logical Diagram

FTL(Flash Translation Layer): Flash Controller 내부에서 동작하는 핵심 소프트웨어

- 뛰어난 성능: Write Amplification 최소화, 여러 Flash 칩 병렬 활용
- 높은 신뢰성: Wear Leveling으로 wear out 주의, Program disturbance 최소화

FTL이란?

→ Flash Translation Layer

논리 블록에 대한 읽기/쓰기 요청을 flash의 physical blocks/pages에 대한 low-level read,erase,program 명령으로 변환하는 계층

나쁜 FTL 방식: Direct Mapped

아이디어: 논리 주소 $N \rightarrow$ 물리 주소 N 에 1:1 대응

쓰기 1번에 발생하는 일:

블록 전체 read

블록 전체 erase

원하는 페이지
program

<성능 문제>

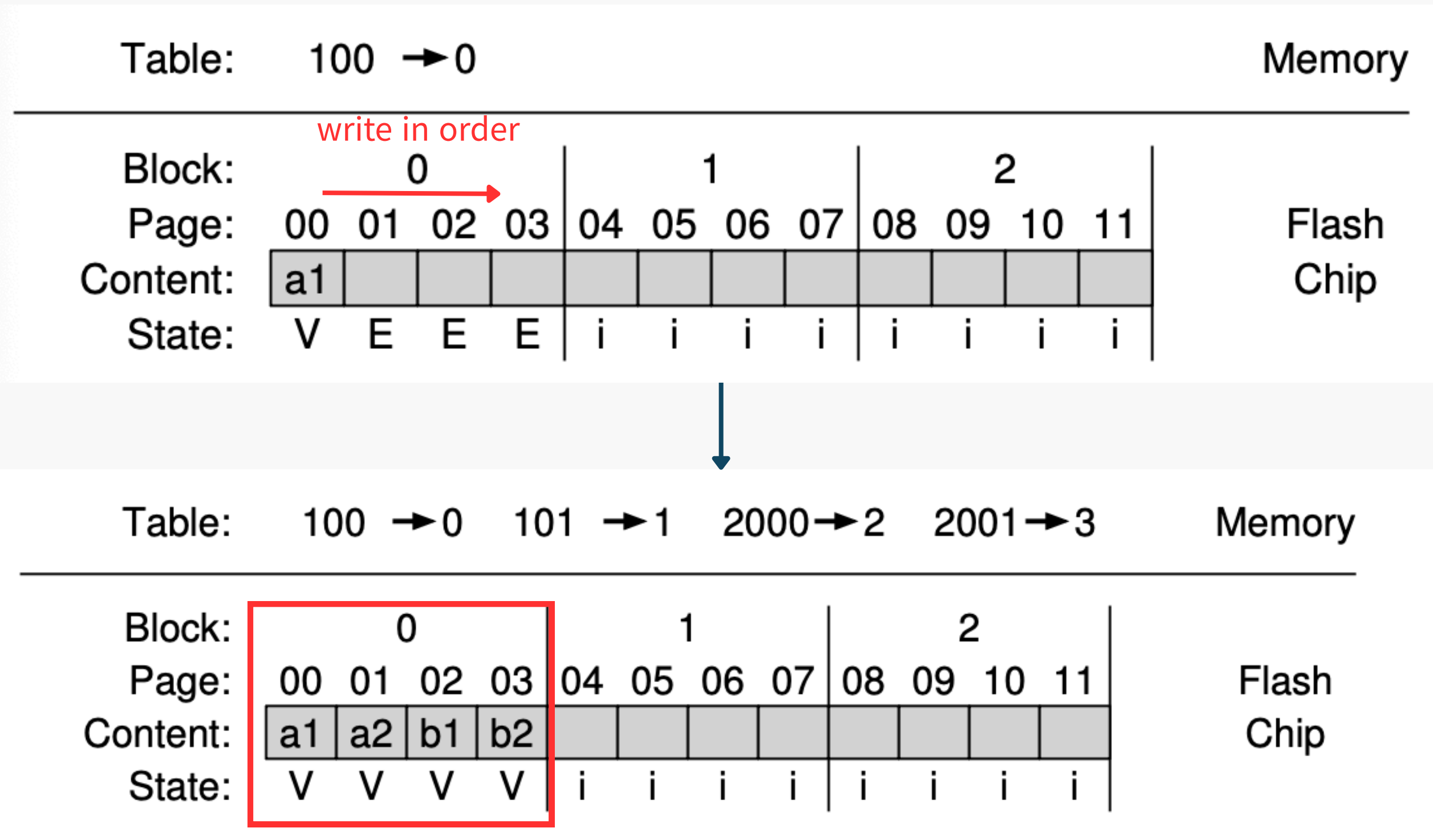
- 각 쓰기마다 전체블록 read, erase, program
- $\rightarrow$ write amplification 극심 $\rightarrow$ 성능 최악

<신뢰성 문제>

- 자주 쓰이는 블록만 wear out

Log-Structured FTL

아이디어: 새 데이터는 항상 log 끝에 append + Mapping table로 주소 관리



Log-Structured FTL

아이디어: 새 데이터는 항상 log 끝에 append + Mapping table로 주소 관리

pros

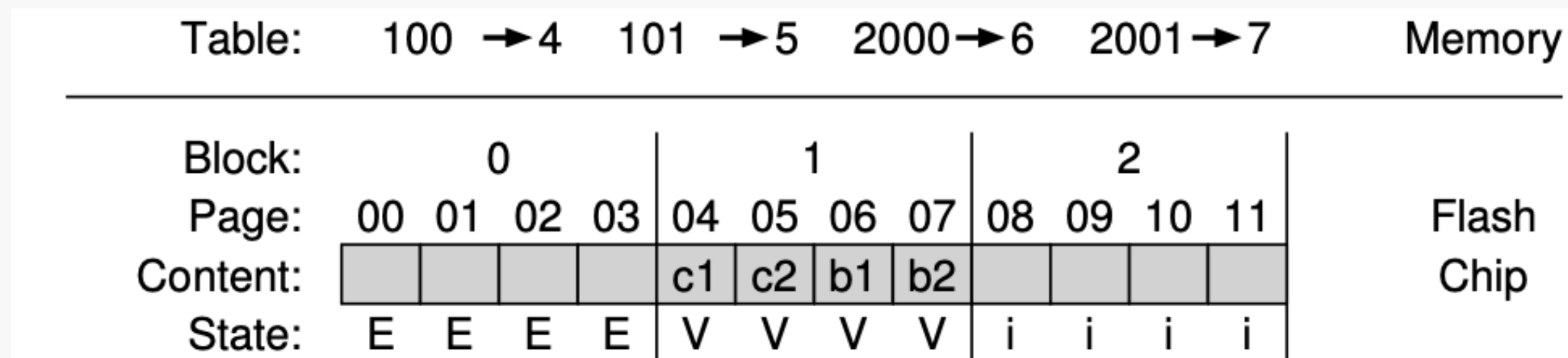
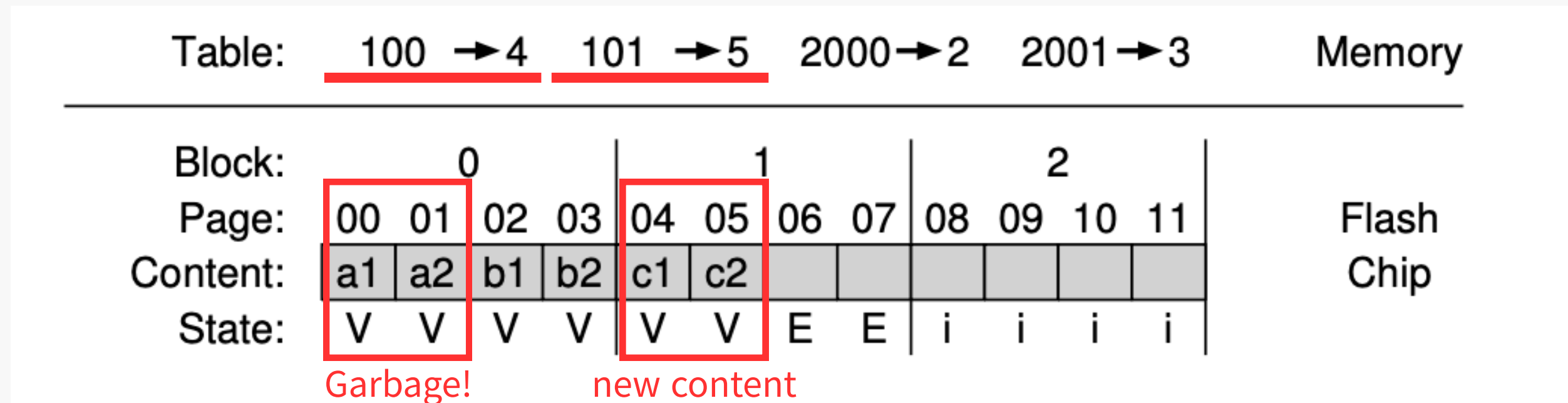
- Erase 횟수 감소
- Write in order → program disturbance 감소
- 자연스러운 wear leveling 효과

cons

- 논리 블록 덮어쓰기로 인한 grabage 생성
 - → write amplification 증가 → 성능 저하
- in-memory mapping table의 높은 비용

Garbage Collection

🚧 Log-structured 방식에서 덮어쓰기 발생 시, Garbage가 물리 공간을 차지
→ 주기적인 청소 필요



GC complete

Garbage Collection

 Log-structured 방식에서 덮어쓰기 발생 시, Garbage가 물리 공간을 차지
→ 주기적인 청소 필요

cons

- live data를 읽고 다시 써야 하므로 비용이 클 수 있음

최적화 방법

- TRIM 인터페이스 사용
- 일부 장치는 overprovision

Mapping table Size

 Log-structuring에서 사용하는 mapping table의 크기가 매우 커질 수 있음!

Block-Based Mapping

- 블록 당 포인터만 유지
- 주소 매핑 방식:
chunk num + offset
- read 간단, write 복잡
- small write 발생 시
성능 ⬇️

Hybrid Mapping

- SOTA
- log block: erase된 상태를 유지하는 블록
- log block에 모든 write를 보냄
- 논리적으로 메모리에 두 종류의 매핑 테이블

Page Mapping + Caching

- 페이지 단위 매핑 유지하되 자주 쓰는 것만 메모리에 캐싱
- 워크로드 locality가 좋으면 효과적

Hybrid Mapping

핵심: log block의 수를 적게 유지하기!

두 종류의
mapping table

log block의 수를
적게 유지하기 위한
3가지 방식

- **log table:** 소수 page 단위 mapping
- **data table:** 더 큰 block 단위 mapping
- **Switch merge:**
 - log block에 덮어쓰는 순서가 data block과 완전 동일한 경우
 - 포인터만 교체
- **Partial merge:**
 - 일부 페이지만 덮어쓰는 경우
 - Write amplification 증가
- **Full merge<worst>:**
 - 여러 블록에서 데이터 끌어모아야 하는 경우 → 가장 비쌘

Wear Out & Wear Leveling

Erase/Program 반복

→ 트랜지스터에 전하 축적

→ 0과 1 구분 불가

→ **Wear Out**



Wear Leveling

: Erase/Program cycle을

모든 블록에 걸쳐 최대한

고르게 분산하는 방법

!! 블록이 덮어쓰여지지 않는 오래 지속되는 데이터로 가득 차는 경우 GC는 해당 블록을 절대 회수하지 않음 **!!**

→ FTL이 주기적으로 cold data를 다른 블록으로 강제 이동

→ 해당 블록을 write 대상으로 돌림

cons: 불필요한 이동이 생기므로 Write amplification 

SSD vs HDD: 성능&비용

HDD

GB 당 5센트
“저장”

→ cold data 저장

SSD

GB 당 60센트
“성능”

→ hot data 저장

- SSD의 성능은 HDD를 크게 능가
- 하지만 SSD가 완전히 HDD를 대체하지 못한 이유 → **비용**



Thank you

